

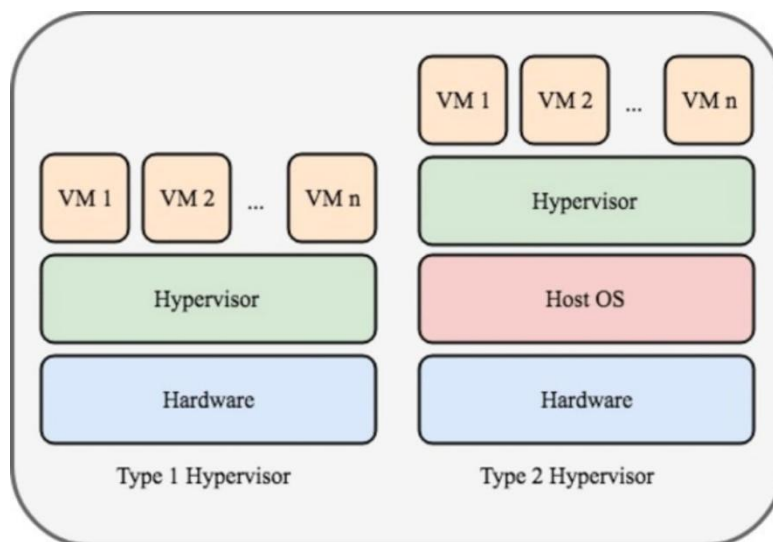
➤ There are 2 types of **Hypervisor**

⌘ 1. Type-1 Hypervisor

- ⌘ Here, Bottom layer is Hardware, then Hypervisor, then multiple Virtual Machines
- ⌘ There is no Host OS in this case. You can imagine as dual-boot as having Type-1 hypervisor.

⌘ 2. Type-2 Hypervisor

- ⌘ Here, bottom layer is Hardware, then Host OS, then Hypervisor then multiple VMs.
- ⌘ Ex: Oracle Virtualbox, you can install it on windows (for example) and lunch multiple virtual machines



➤ **OS** consists of 2 things (in case of no container)

⌘ **Kernel**

- ⌘ It is the **core part** which talks to actual hardware.

⌘ **User space**

- ⌘ It is containing of the **systemd, shell, libraries, applications** (like node, java ..etc)

➤ When we hit **node server.js** the following steps are executed:

⌘ *System call*

⌘ *Kernel*

⌘ *CPU / Memory / Disk*

➤ Containers

➤ What happens when we run some application or execute some commands?

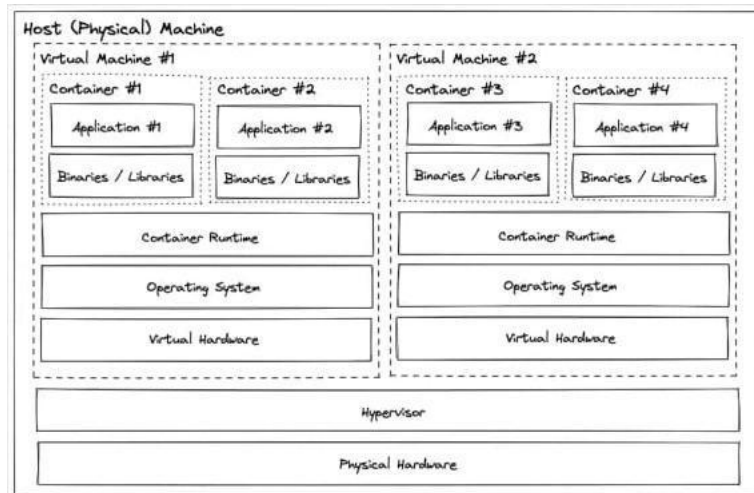
- ⌘ The kernel loads the binaries first.
- ⌘ Lets say you execute **node server.js** then **Kernel** will load the binaries; because Kernel doesn't know anything but the binaries.
- ⌘ In normal system (having no containers), the application (node.js in our case) is installed at some directory, Kernel will load that.
- ⌘ So, all the thing that Kernel needs the binaries.
- ⌘ So, the idea is, **why to install those dependencies/applications in our system? Just keep it in some other path and inform Kernel to use the binaries from that path.**
 - ⌘ This is the core idea of Containers.

➤ Containers

- ⌘ The containers only contains the *application dependencies*, and there is something called as **Container Runtime** in the VM (VM of host machine where the container will run), that's tells Kernel to use the Binaries from that path instead of finding in its own System.
- ⌘ There is no **Container Runtime** during the running state of containers, its just there to set the **isolation rule before starting** the container.
- ⌘ **Container Runtime doesn't sit between Application and Kernel.**
- ⌘ Each container has its own filesystem with its own binaries and dependencies (Node, Java, Maven, etc.).
- ⌘ When containers run, the kernel executes binaries from each container's own filesystem view, because each process has a different *filesystem namespace*, they cannot see or interfere with each other's dependencies.
 - ⌘ namespace means PID, User etc etc.

Namespace	Flag	Page	Isolates
Cgroup	CLONE_NEWCGROUP	cgroup_namespaces(7)	Cgroup root directory
IPC	CLONE_NEWIPC	ipc_namespaces(7)	System V IPC, POSIX message queues
Network	CLONE_NEWNET	network_namespaces(7)	Network devices, stacks, ports, etc.
Mount	CLONE_NEWNS	mount_namespaces(7)	Mount points
PID	CLONE_NEWPID	pid_namespaces(7)	Process IDs
Time	CLONE_NEWTIME	time_namespaces(7)	Boot and monotonic clocks
User	CLONE_NEWUSER	user_namespaces(7)	User and group IDs
UTS	CLONE_NEWUTS	uts_namespaces(7)	

- Means, the **root user** inside the container might be some non-root user in the system that is running the container.
- Application inside the container thinks there is only one process having PID (assigned to itself) is running in the system, but there might be so many processes having different PID running in the systems.



- The container might be running in the Host only, or any of the VMs running on the Host.
- The VMs are having their own **Kernels** (of course because they are having their Own **OS** and **Kernel** is a part of OS only)
 - And they are having their own set of **Virtual Hardware** that are being assigned to them (like **Memory, CPU** etc)
- What does it mean by **Isolation**? Every application present inside the container are running just like a different application (like **Chrome, Firefox**) right?
 - The containers are not having any dedicated **Kernels**, they share the host kernel.
 - Any 2 containers running doesn't interfere each other, they have their own set of libraries or dependencies.
 - Kernel will use their versions of libraries and run them. So, there is no dependency collision or something like that.

➤ Layers in Container

- ⌘ A Docker image is made of Layers.
- ⌘ Layers are *created line-by-line* from the **Dockerfile**
- ⌘ Layers are **read-only**.
- ⌘ When a container runs, Docker adds one more extra **writable** layer on the top.
- ⌘ Example:

```
FROM ubuntu
RUN apt install nodejs
COPY app.js /app.js
```

(Dockerfile)

- ⌘ This creates layers like
 - * Ubuntu layer → base OS files
 - * Node.js layer → Node installed
 - * App layer → your code
 - * Writable layer → created when container starts
- ⌘ **Why layers exist ?**
 - ⌘ **Speed** → unchanged layers are reused
 - ⌘ **Less storage** → same layers shared by many containers
 - ⌘ **Easy updates** → change only the top layer
 - ⌘ Example: **10 Node.js containers → all share same Node layer, Only app code layer is different**
- ⌘ **Writable layer**
 - ⌘ Containers cannot change image layers
 - ⌘ Any file write goes to the top writable layer
 - ⌘ When container is deleted → data is gone;
- ⌘ That's why **use volumes** to store the data like DB data, logs, uploaded files etc

➤ Layers before & after runtime (IMPORTANT)

- First thing: Dockerfile can create **only one type** of Container (can have multiple containers, but all will be identical) if there is no variables are used inside the Dockerfile

```
ARG NODE_VERSION
FROM node:${NODE_VERSION}

WORKDIR /app
COPY . .
CMD ["node", "index.js"]
```

- ⌘ In this case, depending upon the NODE_VERSION variable different type of containers can be created (it is different)

```
docker build --build-arg NODE_VERSION=16 -t app-node16 .
docker build --build-arg NODE_VERSION=18 -t app-node18 .
docker build --build-arg NODE_VERSION=20 -t app-node20 .
```

- ⌘ You can create different types of containers like this

➤ **How the layers are created and how those are re-used (or shared) ?**

- ⌘ Consider the below Dockerfile

```
# Dockerfile.appB
FROM node:18

WORKDIR /app
COPY package.json .
RUN npm install

COPY . .

CMD ["node", "index.js"]
```

- ⌘ Here I the node version **18** is being used.
- ⌘ Now you build this docker file using **docker build** command, Docker will create all the layers.
- ⌘ When we run the docker using **docker run** command.
 - ⌘ It'll create layers from each line of this Dockerfile
 - ⌘ Like Node-18 layer will be present in this case
 - ⌘ And it'll store a **unique hash of each layer**
- ⌘ After that, let's say I executed **docker run** for this Dockerfile again or for some other Dockerfile that contains Node-18 image again
 - ⌘ Now again Docker will scan that Dockerfile, and then see there is a layer already present that have Node-18 (checking the **hash**) and it'll re-use that layer only, without creating another layer.
- ⌘ These all sharing and all are done by **Docker** (not Kernel)
- ⌘ After creating the layers, it'll ask **Kernel** for the namespaces like PID, and all
- ⌘ Then Kernel will run this.

➤ Example of this Layer creation

- ⌘ Consider this Dockerfile again

```
FROM node:18

WORKDIR /app
COPY package.json .
RUN npm install

COPY . .
CMD ["node", "index.js"]
```

- ⌘ Now execute **build** command

```
docker build -t my-node-app .
```

- ⌘ Now **Docker** will create a **IMAGE** layers (**read-only**) (layers are created by **docker**, not **Kernel**)

```
===== IMAGE LAYERS (ON DISK) =====

Layer 5: CMD ["node", "index.js"]      (metadata)
Layer 4: COPY . .                      (app source code)
Layer 3: RUN npm install                (node_modules)
Layer 2: COPY package.json .
Layer 1: WORKDIR /app
Layer 0: node:18 base image
      ├── Debian base
      └── Node.js v18 binaries
```

- ⌘ These layers are stored **once**, identified by **hashes**, **read-only**.
- ⌘ Now execute **run** command

```
docker run --name container-A my-node-app
docker run --name container-B my-node-app
```

- ⌘ Now the layers in the runtime

```
===== CONTAINER A =====

[ Writable Layer A ]      ← unique (logs, temp files)
-----
[ Layer 4 ] App source    ← shared
[ Layer 3 ] node_modules  ← shared
[ Layer 2 ] package.json  ← shared
[ Layer 1 ] /app WORKDIR  ← shared
[ Layer 0 ] node:18       ← shared
```

```

===== CONTAINER B =====

[ Writable Layer B ]      ← unique (logs, temp files)
-----
[ Layer 4 ]  App source    ← shared
[ Layer 3 ]  node_modules ← shared
[ Layer 2 ]  package.json ← shared
[ Layer 1 ]  /app WORKDIR ← shared
[ Layer 0 ]  node:18       ← shared

```

- ☛ All the layers were present, so it is shared now
- ☛ Even if the layers are shared among the containers, but the namespaces (PID, network etc etc) are different for all the containers. Also the **writable-layer** is unique for all containers.
- As container uses the same Kernel of its host, so the host should be Linux as well. Then How are we able to run containers in our Windows or Mac?
 - ☛ Docker desktop create a Linux VM underlying and docker containers run on top of it.
- Name spaces, control groups (cgroups), union filesystems //TODO

- You can run the containers from docker hub like below

- ♣ `docker run --env POSTGRES_PASSWORD=foobabaz --publish 5431:5432 postgres:18.1-alpine`

- As the data is not persisted in the container, because after destroying the containers all the data will be lost, we can use 2 types of mount

- ♣ **Volume mount**

- ♣ Here one **docker volume** is created and mounted to the container

- ♣ **Bind mount**

- ♣ Here, one directory of the **host machine** is mounted to the container.

- ♣ It is not recommended because its crossing the boundary of container.

- Consider the below command

- ♣ `docker run --interactive --tty --rm ubuntu:22.04`

- ♣ **docker run** ---- create + start a container

- ♣ **--interactive** ---- keeps STDIN open (you can type commands)

- ♣ **--tty** ---- allocates a terminal (nice shell experience)

- ♣ **--rm** ---- automatically deletes the container when you exit

- ♣ **--interactive --tty** can be reduced to **-it**

- Every time you run the container, it'll be having a different container ID

- ♣

```
root@59bca493b1fc:/# exit
exit

alokr@Alok MINGW64 /c/my files/devops/docker
$ docker run --interactive --tty --rm ubuntu:22.04
root@5e223d44c15a:/# exit
exit
```

- See the below steps

- ♣ I now ran one container without giving **rm** and giving one name

- ♣ `docker run --interactive --tty --name my-ubuntu-container ubuntu:22.04`

- ♣ Then I installed **iputils-ping** to try to ping any server.

- ♣ `apt install iputils-ping -y`

- ♣

```
root@118e8dbcd4c4:/# ping google.com -c 4
PING google.com (142.250.194.238) 56(84) bytes of data.
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=1 ttl=63 time=28.1 ms
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=2 ttl=63 time=31.9 ms
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=3 ttl=63 time=26.6 ms
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=4 ttl=63 time=28.3 ms
```

- ♣ Then exited using **exit** command

- Now the container will not be deleted after executing **exit** command because we didn't give **--rm** flag during running the container.
- Now, if you do **docker ps -a** it'll show you the container; **docker ps** will not show because the container is stopped.

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
118e8dbcdec4   ubuntu:22.04   "/bin/bash"   3 minutes ago   Exited (0) 23 seconds ago   my-ubuntu-container
bbe4f19b7c6d   postgres:18.1-alpine   "docker-entrypoint.s..."   21 minutes ago   Exited (0) 13 minutes ago   condensing_poincare
b24e5369e888   mysql:latest   "docker-entrypoint.s..."   3 weeks ago   Exited (255) 33 hours ago   mysql
48cf3c282778   postgres   "docker-entrypoint.s..."   3 weeks ago   Exited (255) 33 hours ago   lovable-postgres
```

- Even if this container is stopped, but its still having that **read & write** layers associated with it.
- You can also execute: **docker container ls** or **docker container ls -a**
- Now to start the container, execute the command **docker start my-ubuntu-container**

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker start my-ubuntu-container
my-ubuntu-container
```

- Now **docker ps** (without -a) will show this container

```
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
118e8dbcdec4   ubuntu:22.04   "/bin/bash"   7 minutes ago   Up About a minute   my-ubuntu-container
```

- Now, to open the *interactive shell* execute the command **docker attach my-ubuntu-container**

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker attach my-ubuntu-container
root@118e8dbcdec4:/#
```

- Now you don't need to install *iputils-ping* again, because it is already there and you just started the previously created container which was stopped (not destroyed)
- Here, we ran the container, then installed the *iputils-ping* package and did the things,
- But its not a good practice to run the container and then install the dependencies and all; its better to build the image that contains everything.

```
docker build --tag my-ubuntu-image -<<EOF
FROM ubuntu:20.04
RUN apt update && apt install iputils-ping --yes
EOF
```

- ^ You can build the image like this. (EOF part is nothing but the contents that are written in Dockerfile)
- If you execute **docker image ls** now, you'll be able to see the built image

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
my-ubuntu-image:latest	182380ba5c32	210MB	67.7MB	
mysql:latest	fe036967257b	1.29GB	293MB	U
postgres:18.1-alpine	b40d931bd0e7	403MB	113MB	U
postgres:latest	38d5c9d52203	649MB	168MB	U
ubuntu:22.04	104ae83764a5	119MB	31.5MB	U

- ^
- Now if you run a container using this built image, you will not be needed to install *iputils-ping* image again, it'll be there already

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker run -it --rm my-ubuntu-image
root@181db7d8bdb4:/# ping google.com -c 3
PING google.com (142.250.194.238) 56(84) bytes of data.
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=1 ttl=63 time=28.5 ms
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=2 ttl=63 time=27.1 ms
64 bytes from 142.250.194.238 (142.250.194.238): icmp_seq=3 ttl=63 time=26.4 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 26.419/27.348/28.501/0.864 ms
```

- Create a volume using the command **docker volume create <volume name>**

```
alokr@Alok MINGW64 /c/my files/devops/docker
$ docker volume create my-docker-volume
my-docker-volume
```

- Now run a container and mount that volume to a particular path of container.

- The command will be: **--mount source=<volume name>,destination=<path which to be mounted>**

```
docker run -it --rm --mount source=my-docker-volume,destination=/my-data ubuntu:22.04
```

- It'll automatically create ont directory /my-data and now it is mounted to the volume that I created.

```
root@ec81d6ee409c:/# ls / | grep my-data
my-data
```

- I just created one file inside that directory /my-data

```
root@ec81d6ee409c:/my-data# echo "welcome to the container" > f.txt
root@ec81d6ee409c:/my-data# ls
f.txt
root@ec81d6ee409c:/my-data# cat f.txt
welcome to the container
root@ec81d6ee409c:/my-data# pwd
/my-data
```

- Now exit, so that the container will be deleted (--rm was used while creating the container)

- Now, I created one more container, and now gave the mount destination path as “/some-dir”

```
PS C:\my files\devops\docker> docker run -it --rm --mount source=my-docker-volume,destination=/some-dir ubuntu:22.04
root@231db463fcfa:/# ls / | grep some-dir
some-dir
root@231db463fcfa:/# cd /some-dir/
root@231db463fcfa:/some-dir# ls
f.txt
root@231db463fcfa:/some-dir# cat f.txt
welcome to the container
```

- Now we can see the file is present in that “/some-dir” directory.

- I created one directory “my-data” in my pwd in my system (not in container)

```
PS C:\my files\devops\docker> ls

Directory: C:\my files\devops\docker

Mode                LastWriteTime         Length Name
----                -
d-----          12-01-2026   10:49             my-data
-a----          12-01-2026    586872 docker.docx

PS C:\my files\devops\docker> cat .\my-data\f1.txt
hello from the container
```

- Now I'll bind this directory (**bind mount**) to a container

- ⌘ The command will be: **--mount type=bind,source=<source dir path>,destination=<dir path inside container>**

```
PS C:\my files\devops\docker> docker run -it --rm --mount type=bind,source="$PWD"/my-data,destination=/my-data ubuntu:22.04
root@82cab1876c2d:/# ls /
bin boot dev etc home lib lib32 lib64 libx32 media mnt my-data opt proc root run sbin srv sys tmp usr var
root@82cab1876c2d:/# cat /my-data/f1.txt
hello from the container
```

- ⌘ If you create something inside that “my-data” directory inside the container, then the same file will also be there inside the system’s directory.
- There is another way of mounting volumes or binding the directory to the container, that is **-v** flag

```
alokr@Alok MINGW64 /c/my files/devops/docker/devops-directive
$ docker run -it --rm -v test_volume:/tmp/test ubuntu:22.04
```

- ⌘ If the volume is there then it'll bind that; otherwise **it'll create one and bind that.**

➤ Writing Dockerfile

➤ The following instructions are supported in Dockerfile

➤ ADD	Add local or remote files and directories.
➤ ARG	Use build-time variables.
➤ CMD	Specify default commands (after building container).
➤ COPY	Copy files and directories.
➤ ENTRYPOINT	Specify default executable.
➤ ENV	Set environment variables.
➤ EXPOSE	Describe which ports your application is listening on.
➤ FROM	Create a new build stage from a base image.
➤ HEALTHCHECK	Check a container's health on startup.
➤ LABEL	Add metadata to an image.
➤ MAINTAINER	Specify the author of an image.
➤ ONBUILD	Specify instructions for when the image is used in a build.
➤ RUN	Execute build commands (during building container).
➤ SHELL	Set the default shell of an image.
➤ STOPSIGNAL	Specify the system call signal for exiting a container.
➤ USER	Set user and group ID.
➤ VOLUME	Create volume mounts.
➤ WORKDIR	Change working directory.

➤ We can write some simple command like below inside the Dockerfile

```
FROM ubuntu  
  
RUN apt update  
  
RUN apt install nodejs -y
```

⚡ As each line will be created as a layer, we should avoid creating multiple layers. So, instead of writing like above, write like the below

```
FROM ubuntu  
  
RUN apt update && apt install nodejs -y
```

➤ Below are the stages of Dockerfile from non-optimized to optimized
(for nodejs application)

- In the below case

```
FROM ubuntu

RUN apt update

RUN apt install nodejs npm -y
```

^

- ^ Writing RUN in different lines creates 2 different layers; we should minimize the number of layers to better to write in a single line.

- Stage-1 (building image tht runs nodejs applicatoin):

```
FROM ubuntu

RUN apt update && apt install nodejs npm -y

COPY . .

RUN npm install

CMD ["npm", "run", "dev"]
```

^

```
FROM ubuntu

RUN apt update && apt install nodejs npm -y

COPY ./ ./

RUN npm install

CMD ["npm", "run", "dev"]
```

^

- ^ Both are same; `.` is same as `./` in this COPY command.

- `docker build -t api-node:0 .` is used to create a image giving a tag

- ^ That dot `.` at the end means use the default docker file which is **Dockerfile**

- ^ Otherwise you can give the dockerfile name

- ^ **-t image-name:tag**

```
$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
api-node:0	b2ca5d8a15ea	1.57GB	363MB	

^

- ^ If you see here, the image is of 1.5 gb which is terrible.

- **Stage-2** : used **node** container image instead of **ubuntu**

```
FROM node

COPY . .

RUN npm install

CMD ["npm", "run", "dev"]
```

^

IMAGE	ID	DISK USAGE	CONTENT SIZE
api-node:0	b2ca5d8a15ea	1.57GB	363MB
api-node:1	44df266483cc	1.7GB	420MB

^

- ^ Now it is also huge; because **node** means it'll find the latest node image and build using that.

- **Stage-3** : used **alpine** version of **node** instead of default (latest)

```
FROM node:25.2-alpine3.23

COPY . .

RUN npm install

CMD ["npm", "run", "dev"]
```

^

IMAGE	ID	DISK USAGE	CONTENT SIZE
api-node:0	b2ca5d8a15ea	1.57GB	363MB
api-node:1	44df266483cc	1.7GB	420MB
api-node:2	644a1d583af7	312MB	71.9MB

^

- ^ Now it is reduced to **312mb**

- **Stage-3** : till now we didn't use the caching due to which build time was very high

```
FROM node:25.2-alpine3.23

COPY package*.json .

RUN npm install

COPY . .

CMD ["npm", "run", "dev"]
```

^

- ^ You might be thinking, this is same only; anyways all the files are being copied as we have written **COPY . .** after running **npm install** command.

- ⌘ Dockerfile instructions are executed **top to bottom**
- ⌘ Each instruction creates a new immutable layer
- ⌘ Layers are **linear and dependent** on previous layers
- ⌘ During build, Docker checks cache for each layer
- ⌘ Cache validation is based on:
 - ⌘ Instruction content
 - ⌘ Parent layer hash
 - ⌘ Files involved (for COPY / ADD)
- ⌘ If everything matches → **cache hit**
- ⌘ If anything differs → **cache miss**
- ⌘ On a cache miss:
 - ⌘ **That layer is rebuilt**
 - ⌘ **All layers after it are invalidated and rebuilt.**
- ⌘ On doing this:

```
COPY package*.json ./
RUN npm ci
COPY . .
```

- ⌘ npm ci runs only when package*.json changes.
- ⌘ Source code changes do not invalidate dependency cache
- ⌘ Using only:


```
COPY . .
RUN npm install
```

 - ⌘ **Any file change** → cache invalidation
 - ⌘ npm install runs every build

```
=> [1/3] FROM docker.io/library/node:25
=> => resolve docker.io/library/node:25
=> CACHED [2/3] COPY . .
=> CACHED [3/3] RUN npm install
```

- ⌘ Now you can see it is cached so no need to build again.

- **Stage-4** : you should give **WORKDIR** ,
 - ^ by default when you write **COPY .** it copies to the default work directory.

```
FROM node:25.2-alpine3.23

WORKDIR /usr/src/app

COPY package*.json .

RUN npm install

COPY . .

CMD ["npm", "run", "dev"]
```

- **Stage-4** : maintain **.dockerignore** file to ignore the files

```
.dockerignore M
evops-directive-docker-
1 node_modules
```

- **Stage-5** : as it is a production container, we can run **node index.js** directly.

```
FROM node:25.2-alpine3.23

WORKDIR /usr/src/app

COPY package*.json .

RUN npm install

COPY ./src .

CMD ["node", "index.js"]
```

IMAGE	ID	DISK USAGE	CONTENT SIZE
api-node:0	b2ca5d8a15ea	1.57GB	363MB
api-node:1	44df266483cc	1.7GB	420MB
api-node:2	644a1d583af7	312MB	71.9MB
api-node:3	4e77dd64056e	312MB	71.9MB
api-node:4	9ae2ecf21756	312MB	72MB
api-node:5	4bdb486c11ff	312MB	71.9MB
api-node:6	13ae5faee0a2	312MB	71.9MB

- Till now we are optimizing w.r.t. build time and image size; now we'll optimize w.r.t. security.
- Stage-6 : execute the things as a **non-root** user.
 - ⌘ In the **node** container image, there is a existing user named as **node**

```
FROM node:25.2-alpine3.23

WORKDIR /usr/src/app

COPY package*.json .

RUN npm install

USER node

COPY --chown=node:node ./src .

CMD ["node", "index.js"]
```

⌘ **USER** command just switch to that user; it doesn't create a new user.

- ⌘ I made sure that those files should be owned by the user **node**
- Stage-7 : setting the environment
 - ⌘ Mostly node application like react, express, next ...etc uses **NODE_ENV**.
 - ⌘ **--only=production** is npm's flag; if **--only** is not specified then **npm** checks the **NODE_ENV** otherwise it'll take the value from that **--only** flag only.

```
FROM node:25.2-alpine3.23

WORKDIR /usr/src/app

ENV NODE_ENV=production

COPY package*.json .

RUN npm ci --only=production

USER node

COPY --chown=node:node ./src .

CMD ["node", "index.js"]
```

- So, these below 2 are basically same only

```
RUN npm ci --only=production
```

```
ENV NODE_ENV=production
```

```
RUN npm ci
```

- Now due to *production environment* the dev-dependencies were not installed so image size got reduced.

IMAGE	ID	DISK USAGE	CONTENT SIZE
api-node:0	b2ca5d8a15ea	1.57GB	363MB
api-node:1	44df266483cc	1.7GB	420MB
api-node:2	644a1d583af7	312MB	71.9MB
api-node:3	4e77dd64056e	312MB	71.9MB
api-node:4	9ae2ecf21756	312MB	72MB
api-node:5	4bdb486c11ff	312MB	71.9MB
api-node:6	13ae5faee0a2	312MB	71.9MB
api-node:7	7f2428697559	243MB	59.7MB

➤ **Stage-8 : give EXPOSE port**

- EXPOSE doesn't actually expose any port, it just add metadata just like the documentation.

```
FROM node:25.2-alpine3.23

WORKDIR /usr/src/app

ENV NODE_ENV=production

COPY package*.json .

RUN npm ci --only=production

USER node

COPY --chown=node:node ./src .

EXPOSE 3000

CMD ["node", "index.js"]
```

➤ **Stage-9 : caching using DOCKER_BUILDKIT**

⌘ In case of legacy build, if the cache is invalidated then it'll be build again.

⌘ Take the below code:

```
COPY package*.json .  
  
RUN npm ci --only=production
```

⌘ If the package.json is changed then it'll download all the dependencies again.

⌘ Lets say we have added one more dependency in package.json, it should download that new one only and because remaining are same, it should take those from cache only.

⌘ But, legacy build was not working on this. It will re-download all those dependencies again from the beginning.

⌘ So, DOCKER_BUILDKIT solves this problem, it'll only download the new one and the remaining will be taken from the cache.

⌘ **Legacy:**

⌘ **Build 1:** npm downloads everything

⌘ **Build 2:** npm download everything again

⌘ **Buildkit:**

⌘ **Build 1:** npm downloads everything

⌘ **Build 2:** npm downloads ONLY Changed Dependencies.

```
FROM node:25.2-alpine3.23  
  
WORKDIR /usr/src/app  
  
ENV NODE_ENV=production  
  
COPY package*.json .  
  
RUN --mount=type=cache,target=/usr/src/app/.npm \  
    npm set cache /usr/src/app/.npm && \  
    npm ci --only=production  
  
USER node  
  
COPY --chown=node:node ./src .  
  
EXPOSE 3000  
  
CMD ["node", "index.js"]
```

- ⌘ In case of **Buildkit**, the cache is outside of the image layers. Image layers cache are not persistent.
 - ⌘ **--mount=type=cache,target=/usr/src/app/.npm** : create a persistent cache location
 - ⌘ **npm set cache /usr/src/app/.npm** : points npm to that cache
 - ⌘ **npm ci** : download only what missed in cache
 - ⌘ You need to write **DOCKER_BUILDKIT=1** in the command
- ```
⌘ DOCKER_BUILDKIT=1 docker build -t api-node:9 .
```

➤ Below are the stages of Dockerfile from non-optimized to optimized  
(for **golang** application)

- Stage 1 : base config using **golang** latest cont

```
FROM golang

WORKDIR /app

COPY . .

RUN go mod download

CMD ["go", "run", "./main.go"]
```

- Stage 2 : alpine version of the golang image

```
FROM golang:1.25.5-alpine

WORKDIR /app

COPY . .

RUN go mod download

CMD ["go", "run", "./main.go"]
```

| IMAGE        | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------|--------------|------------|--------------|-------|
| api-golang:0 | 1d9203d6e26e | 1.44GB     | 349MB        |       |
| api-golang:1 | e84f4ce5d711 | 525MB      | 109MB        |       |

- Stage 3 :

- In stage 2 we were using **go** compiler (**go run ./main.go**) which is slower & container requires **go** compiler inside it.
- Its better to create binary executable file and run that directly.

```
FROM golang:1.25.5-alpine

WORKDIR /app

COPY . .

RUN go mod download

RUN go build -o api-golang

CMD ["./api-golang"]
```



- It means whenever we are starting our container, we don't need to build the code again, it is already built; just run the binary executable file.

|              |              |        |       |
|--------------|--------------|--------|-------|
| api-golang:0 | 1d9203d6e26e | 1.44GB | 349MB |
| api-golang:1 | e84f4ce5d711 | 525MB  | 109MB |
| api-golang:2 | 690385d14c12 | 759MB  | 155MB |

- It is not possible for nodejs application because *golang* is a **compiled** language where as *Node.js* is an **interpreted** language.
- We need to run **node index.js** in nodejs application, but in golang we can directly execute **./api-golang** which is a compiled binary file.

➤ Stage 4 : optimize the COPY of files

```
FROM golang:1.25.5-alpine

WORKDIR /app

COPY go.mod go.sum ./

RUN go mod download

COPY . .

RUN go build -o api-golang

CMD ["/api-golang"]
```

| IMAGE        | ID           | DISK USAGE | CONTENT SIZE |
|--------------|--------------|------------|--------------|
| api-golang:0 | 1d9203d6e26e | 1.44GB     | 349MB        |
| api-golang:1 | e84f4ce5d711 | 525MB      | 109MB        |
| api-golang:2 | 690385d14c12 | 759MB      | 155MB        |
| api-golang:3 | de5a684b0968 | 759MB      | 155MB        |

➤ Stage 5 : **multi-stage build**

- scratch** is a image that contains nothing.
- We can use one image to build i.e. compile and generate the binary and will transfer this binary to another smaller image.
- in case of **go build -o api-golang** it'll generate the binary executable file but this file will be having dependency with the OS libraries.
- So we'll use **-ldflags** and **-tags** to create a fully static Go binary file.

```
-ldflags="-linkmode external -extldflags '-static'" \
-tags netgo \
-o api-golang`
```



- ⌘ `-linkmode external` ===== Forces use of external linker
- ⌘ `-extldflags '-static'` ===== Links everything statically
- ⌘ `-tags netgo` ===== dns handled by `go` (`netgo`), not `libc`

```
first build
FROM golang:1.19-buster as build

WORKDIR /app

COPY go.mod go.sum ./

RUN go mod download

COPY . .

RUN go build -o api-golang ##---- it'll not work in scratch image
RUN go build \
 -ldflags="-linkmode external -extldflags '-static'" \
 -tags netgo \
 -o api-golang

second (final) build
FROM scratch

COPY --from=build /app/api-golang api-golang

CMD ["/api-golang"]
```

| IMAGE        | ID           | DISK USAGE | CONTENT SIZE |
|--------------|--------------|------------|--------------|
| api-golang:0 | 1d9203d6e26e | 1.44GB     | 349MB        |
| api-golang:1 | e84f4ce5d711 | 525MB      | 109MB        |
| api-golang:2 | 690385d14c12 | 759MB      | 155MB        |
| api-golang:3 | de5a684b0968 | 759MB      | 155MB        |
| api-golang:4 | 40f08eea4728 | 24.5MB     | 7.69MB       |

⌘ If you see now, this container's size is just 24 MB

➤ Stage 5 : DOCKER\_BUILDKIT caching, ENV variable & use EXPOSE

```
first build
FROM golang:1.19-buster as build

WORKDIR /app

COPY go.mod go.sum ./

RUN --mount=type=cache,target=/go/pkg/mod \
 --mount=type=cache,target=/root/.cache/go-build \
 go mod download

COPY . .

RUN go build -o api-golang ##---- it'll not work in scratch image
RUN go build \
 -ldflags="-linkmode external -extldflags '-static'" \
 -tags netgo \
 -o api-golang

second (final) build
FROM scratch

ENV GIN_MODE=release

COPY --from=build /app/api-golang api-golang

EXPOSE 8080

CMD ["/api-golang"]
```

♣ You can cache as many paths as you want.

- Stage 6 : use non-root user for all these

```
first build
FROM golang:1.19-buster as build

WORKDIR /app

RUN useradd -u 10001 nonroot

COPY go.mod go.sum ./

RUN --mount=type=cache,target=/go/pkg/mod \
 --mount=type=cache,target=/root/.cache/go-build \
 go mod download

COPY . .

RUN go build -o api-golang ##---- it'll not work in scratch image
RUN go build \
 -ldflags="-linkmode external -extldflags '-static'" \
 -tags netgo \
 -o api-golang

second (final) build
FROM scratch

ENV GIN_MODE=release

COPY --from=build /etc/passwd /etc/passwd

COPY --from=build /app/api-golang api-golang

USER nonroot

EXPOSE 8080

CMD ["/api-golang"]
```

- Create one user using **RUN useradd** command and copy the */etc/passwd* file from first build to final build.
- 
- F
- F
- F
- F

➤ **F**

➤ **F**

➤ **F**

➤ F

➤ F

➤ **F**

➤ **F**

➤ **F**

➤ **F**

➤ F

➤ **F**